

9.1. #include <multiplier> — a * b, quand les deux se calculent

#include <multiplier> : le processeur ne sait pas multiplier ; a * b, quand ni a ni b n'est écrit en clair, demande une routine.

1. Ce qu'il faut comprendre

Ce que c'est : le processeur de la Game Boy **ne sait pas multiplier**. Fois un nombre écrit en clair (a * 3), le compilateur s'en sort tout seul en additionnant. Mais quand **les deux** se calculent (a * b), il faut une petite routine qui multiplie.

La ligne à écrire : #include <multiplier>. Sans elle, a * b est refusé ; a * 3 passe toujours.

Ce qu'elle coûte : environ **25 octets**.

Essaie : remplace b par 7 et enlève la ligne #include <multiplier> : ça compile encore.

On retrouve multiplier dans 1 leçon

[10](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **403 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	60 o	← la console, à l'allumage — la mise en route de la console
main()	27 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre — attend de pouvoir écrire une case
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Multiplier	14 o	← main() — multiplie — le processeur ne sait pas le faire
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles — 10 lettres sur 43

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

nombre() × 1	10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <multiplier> : a * b, quand les deux se calculent ----  
  
#include <multiplier>    // la routine qui multiplie deux valeurs calculées  
#include <nombre>
```

2. UNE VARIABLE — A

```
uint8_t a = 6;
```

3. UNE VARIABLE — B

```
uint8_t b = 7;
```

4. UNE VARIABLE — PRODUIT

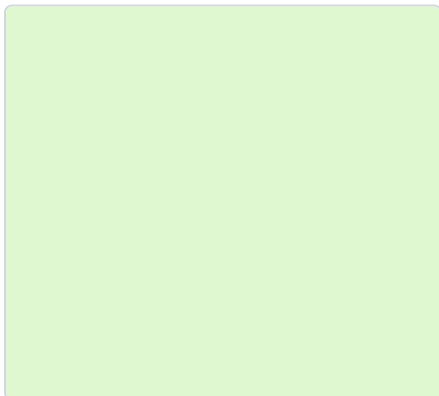
```
uint8_t produit = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    produit = a * b;           // 6 × 7, calculé pendant le jeu  
    nombre(8, 8, produit);    // 042  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

042 au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ $6 \times 7 = 42$

✓ 042 s'affiche

Ensuite : **10. Calculer : + - * / et %** — chapitre 2 : Les variables : la mémoire de la console. · [gameboy3](#) — écrire une cartouche Game Boy en C++